Step 2 — Extract an Invoice From Text (the model's job) — Solution

> Back to the step · Hint 1 · Hint 2

Story: the one place the model is involved — turn raw invoice text into a typed `Invoice` with `with_structured_output`, the guaranteed-shape method from Doc04/05. **If not:** hand-parsing the model's text into fields is exactly the fragile "please return JSON" problem structured output exists to remove.

The extractor

Start `src/pipeline.py` with the prompt and the extractor. The prompt's one job is to stop the model inventing values for fields that aren't in the document.

```
# src/pipeline.py  (Step 2 version — the extractor)
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import Invoice

EXTRACT_PROMPT = (
    "Extract the invoice fields from the text below. If a field is not "
    "present, leave it empty - never invent a value.\n\nInvoice text:\n{text}"
)


def extract_invoice(model, text):
    prompt = ChatPromptTemplate.from_template(EXTRACT_PROMPT)
    chain = prompt | model.with_structured_output(Invoice)
    return chain.invoke({"text": text})
```

That's the whole step. `model.with_structured_output(Invoice)` makes the model return an `Invoice` object directly — no JSON string, no parsing. Because every field on `Invoice` is optional (Step 1), a field the document doesn't mention comes back as `None` or `[]`, which your validator then reports as a problem — instead of the model guessing a plausible-looking number.

Why this is the easy half

- **Structured output guarantees the shape, not the truth.** You get a real `Invoice` every time; whether its `total` is *correct* is the validator's job (Step 1), not the model's.
- **The prompt matters in one way only:** "leave it empty, never invent." Without that line, a required-but-missing field tempts the model to fill in something believable, which is the worst kind of wrong — it looks fine.
- **No parsing, no `try/except` around JSON.** The fragile part of the old raw-SDK way is simply gone.

Checking it quickly

You can confirm `extract_invoice` returns an `Invoice` on a hand-written sample without reading a file yet (the loader is Step 3):

```
sample = ("Vendor: Acme\nInvoice Number: INV-1001\n"
          "3 x Widget @ 10.00\nTotal: 30.00")
invoice = extract_invoice(model, sample)
print(type(invoice).__name__, invoice.invoice_number, invoice.total)
# -> Invoice INV-1001 30.0
```

The full, live check (a real file through the whole pipeline) is the integration test in Step 3.